

Software Engineering

Tahir Farooq
FAST-NU

Last Lecture Review

- SQA Reviews
 - Objectives of Reviews
 - Kinds of Reviews
- Responsibilities of Roles
 - Facilitator
 - Author
 - Recorder
 - Reviewer
 - Observer

What Will You Learn Today?

- Introduction to software testing
- Definitions and process
- Testing approaches
- Unit testing

Why do we do testing?

What is testing?

Who does testing?

What are levels of testing?

What are different methods/techniques of testing?

How to test a software?

Common software problems!

What are the problems you encounter as software users?

Incorrect calculation

Incorrect data edits & ineffective data edits

Incorrect matching and merging of data

Data searches that yield incorrect results

Incorrect processing of data relationship

Incorrect coding/processing of business rules

Inadequate software performance

Confusing or misleading data

Inconsistent processing

Unreliable results or performance

Incorrect or inadequate interfaces with other systems

Incorrect file handling

Who is responsible for testing?

Objectives of a software tester

- Find bugs as **early as possible** and make sure they get fixed
- To **understand** the application well
- Study the functionality in detail to find **where the bugs** are likely to occur.
- Study the code to **ensure** that each and every line of **code is tested**.
- Create test cases in such a way that testing is done to uncover the hidden bugs and also ensure that the software is usable and reliable

Objectives of testing

- Executing a program with the intent of finding an *error*
- To check if the system meets the requirements and be executed successfully in the Intended environment
- To check if the system is “Fit for purpose”
- To check if the **system does** what it is **expected** to do
- A **good test case** is one that has a probability of finding an as yet undiscovered error
- A good test is not redundant
- A good test should neither be too simple nor too complex

Confusing but related!!!

Verification (*Are the algorithms coded correctly?*)

- The set of activities that ensure that software correctly implements a specific function or algorithm
- Involves reviews and meeting to evaluate documents, plans, code, requirements, and specifications.
- Usually done with checklists, walkthroughs, and inspection meeting.

Validation (*Does it meet user requirements?*)

- The set of activities that ensure that the software that has been built is traceable to customer requirements
- Involves actual testing and takes place after verifications are completed.

Validation and Verification process continue in a cycle till the software becomes defects free.

INTRODUCTION SOFTWARE TESTING

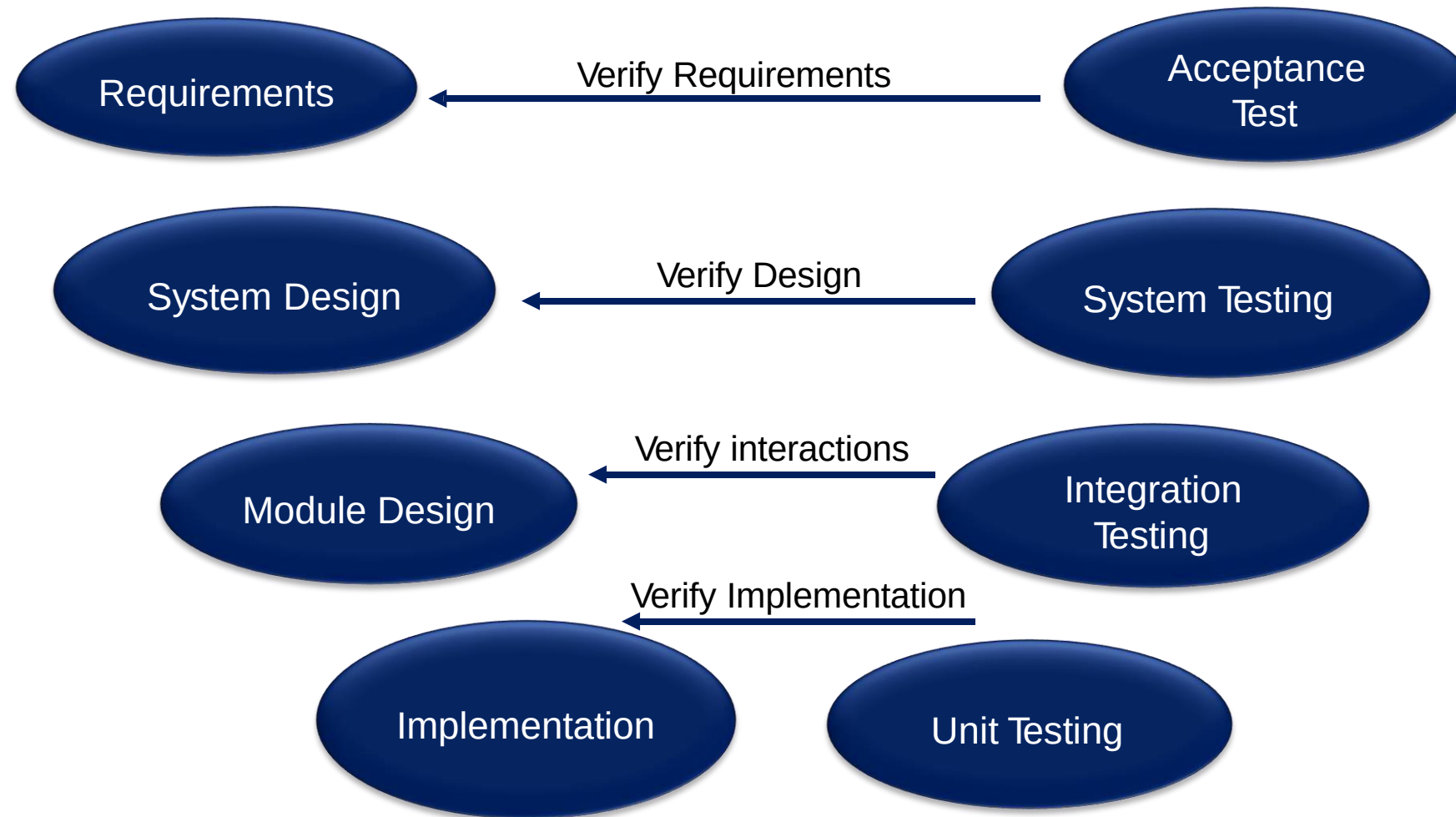
What is Software Testing?

- A process of analyzing a software item to detect the differences between existing and required conditions (that is defects / errors / bugs) and to evaluate the features of the software item

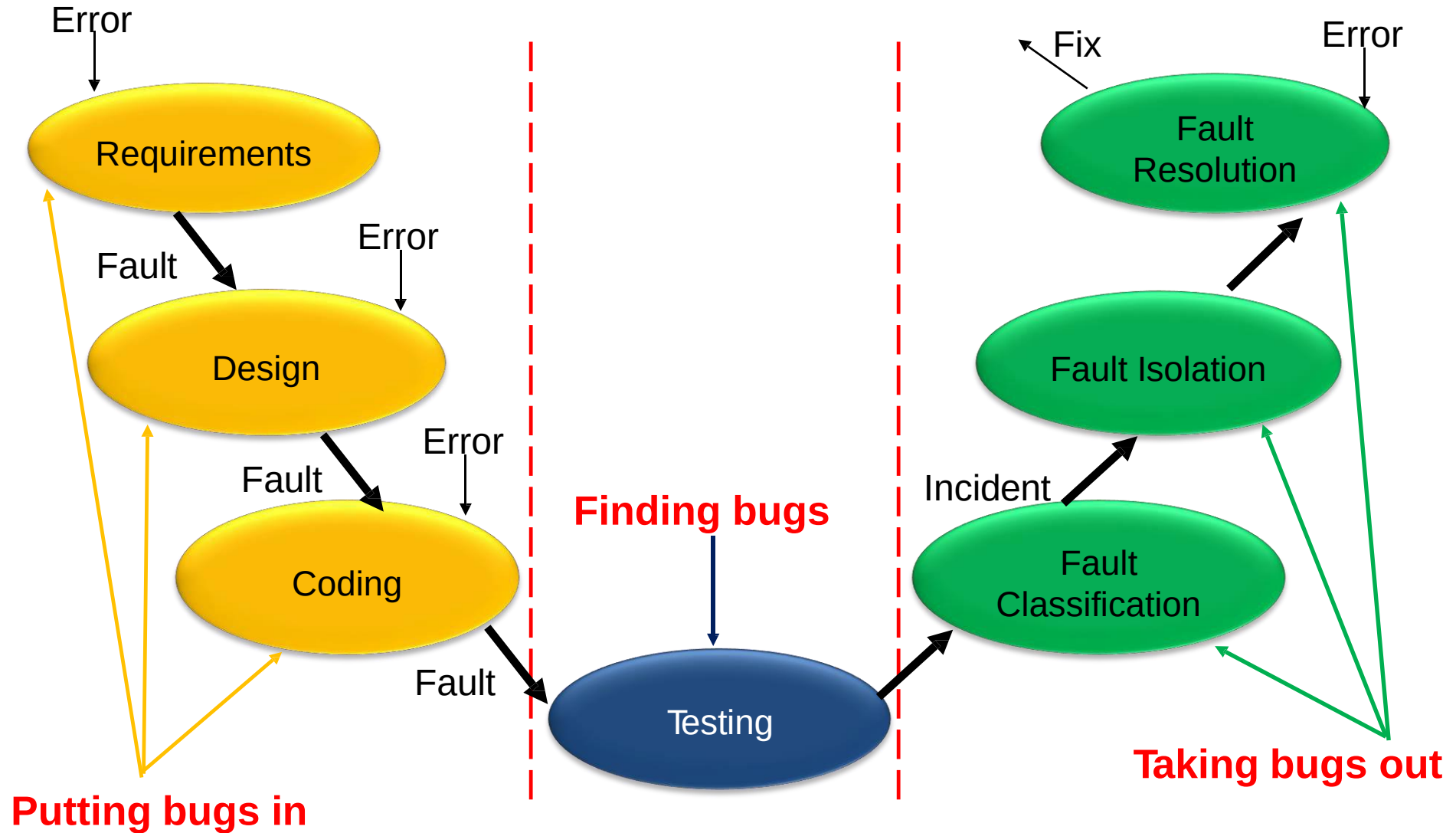
[IEEE]

DEFINITIONS AND PROCESS

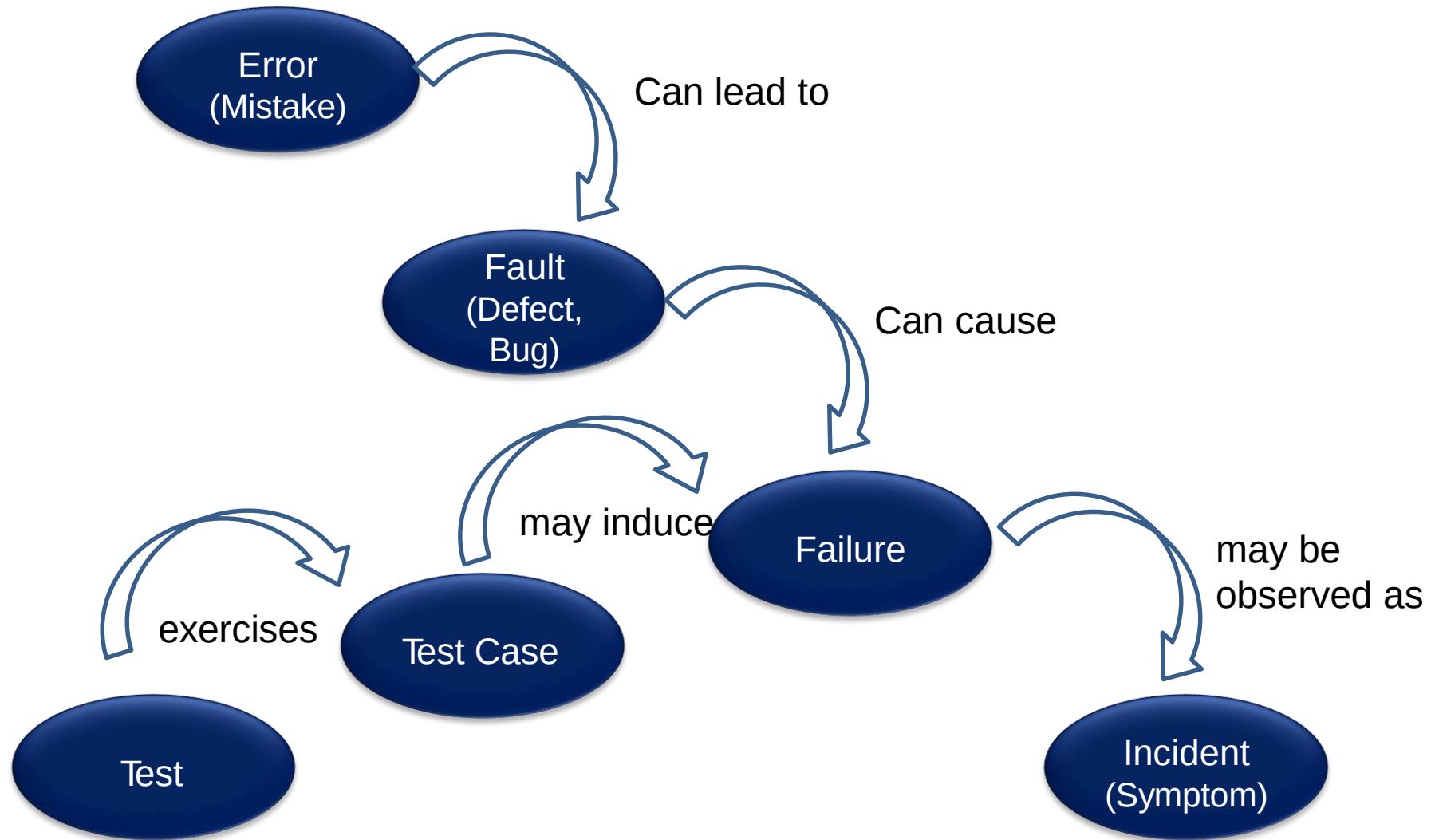
A Software Life-Cycle Model



A Testing Life-Cycle Model



The Tester's Language



Basic Definitions – Error

- People make **errors**
- A good synonym is **mistake**
- When people make mistakes while **coding**, we call these mistakes **bugs**
- **Errors** tend to **propagate**; a requirements error may be magnified during design and amplified still more during coding

[IEEE]



Error (Mistake)

Basic Definitions – Fault

- A **fault** is the **result** of an **error**
- **Defect** is a good synonym for fault, as is **bug**
- When a **designer** makes an **error of omission**, the resulting fault is that **something is missing** that should be present in the representation
- We might speak of faults of **commission** and faults of **omission**

[IEEE]



Fault
(Defect, Bug)

Basic Definitions – Failure

- A **failure** occurs when a **fault executes**
- Two subtleties arise here
 - 1) **Failures only occur** in an **executable** representation, which is usually taken to be source code, or more precisely, loaded object
 - 2) This definition relates failures only to **faults of commission**

[IEEE]



Fault
(Defect, Bug)

Basic Definitions – Incident

- When a **failure occurs**, it **may or may not** be readily **apparent** to the user (or customer or tester)
- An **incident** is the **symptom** associated with a **failure** that alerts the user to the occurrence of a failure



[IEEE]

Basic Definitions – Test

- Testing is obviously concerned with errors, faults, failures, and incidents
- "A test is the act of exercising software with test cases"
 - A test has two distinct goals:
 - 1) To find failures or
 - 2) Demonstrate correct execution

[IEEE]



Basic Definitions – Test Case

- Test case has an **identity** and is **associated with a program behavior**
- A test case also has a **set of inputs** and a **list of expected outputs**

[IEEE]



Contents of a Test Case

- "Boilerplate": Author, Date, Purpose, Test Case ID
- Pre-Conditions (including environment)
- Inputs
- Expected Outputs
- Observed Outputs



[IEEE]

S/W Testing lifecycle phases

- Requirements study
- Analysis and planning
- Test Case Design and Development
- Test Execution
- Test Closure
- Test Process Analysis

Requirements study

- Testing Cycle starts with the study of client's requirements.
- Understanding of the requirements is very essential for testing the product – Why?.

Analysis & Planning

- Test **objective** and **coverage**
- Overall **schedule**
- **Standards** and **Methodologies**
- **Resources** required, including necessary **training**
- **Roles** and **responsibilities** of the team members
- **Tools** used

Test Case Design and Development

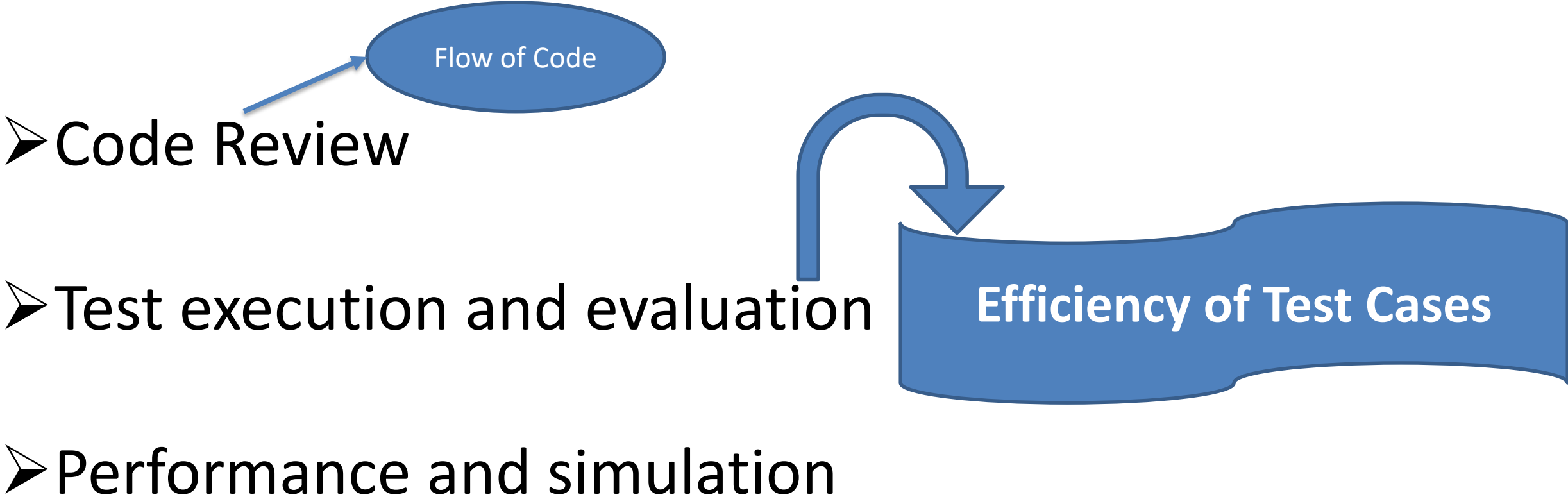
- Component Identification
- Test Specification Design
- Test Specification Review

**To Maximize
the Coverage**

Test Execution

- Code Review
- Test execution and evaluation
- Performance and simulation

Test Execution



Flow of Code

➤ Code Review

➤ Test execution and evaluation

Efficiency of Test Cases

➤ Performance and simulation

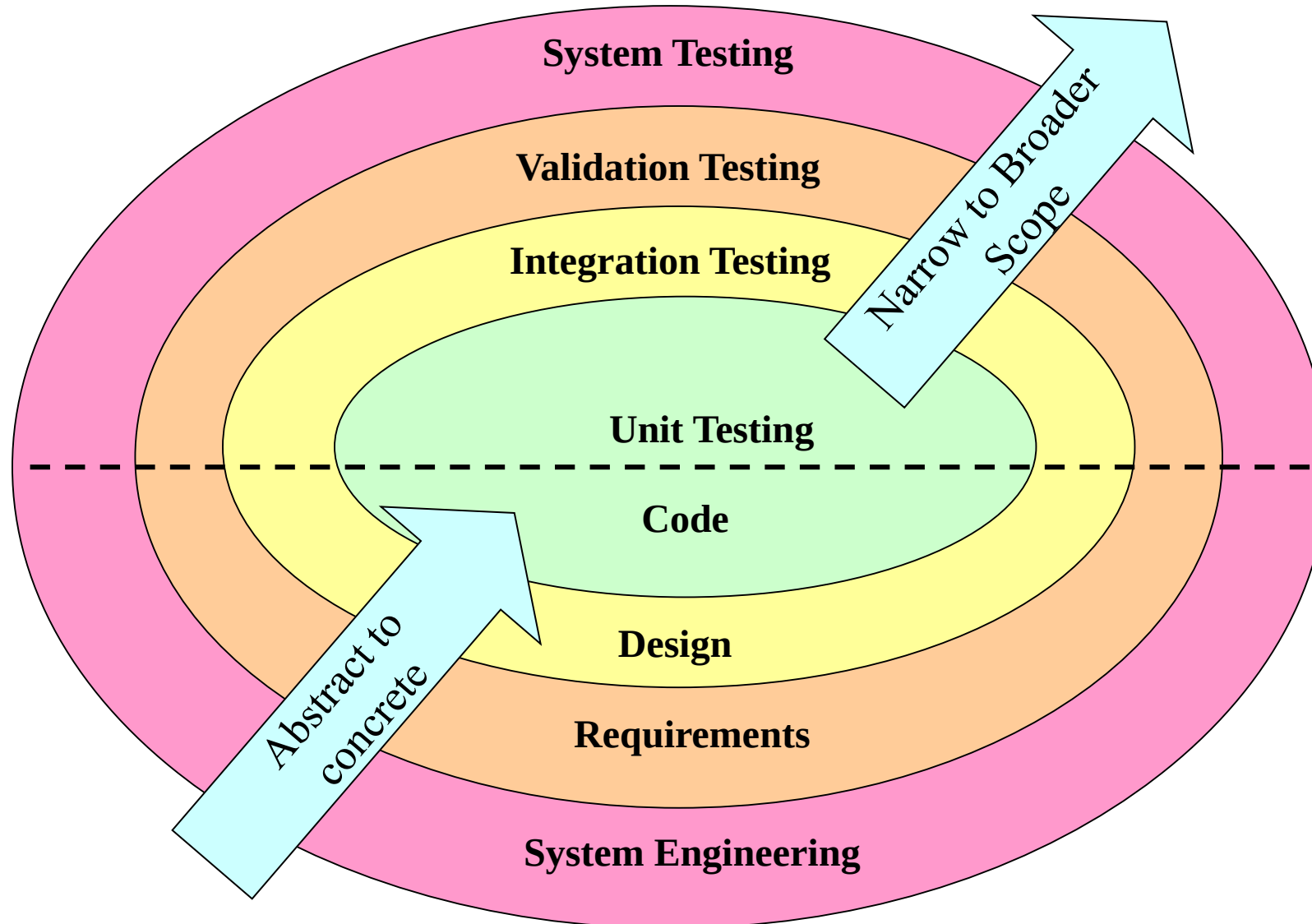
Test Closure

- Test summary report
- Project De-brief
- Project Documentation

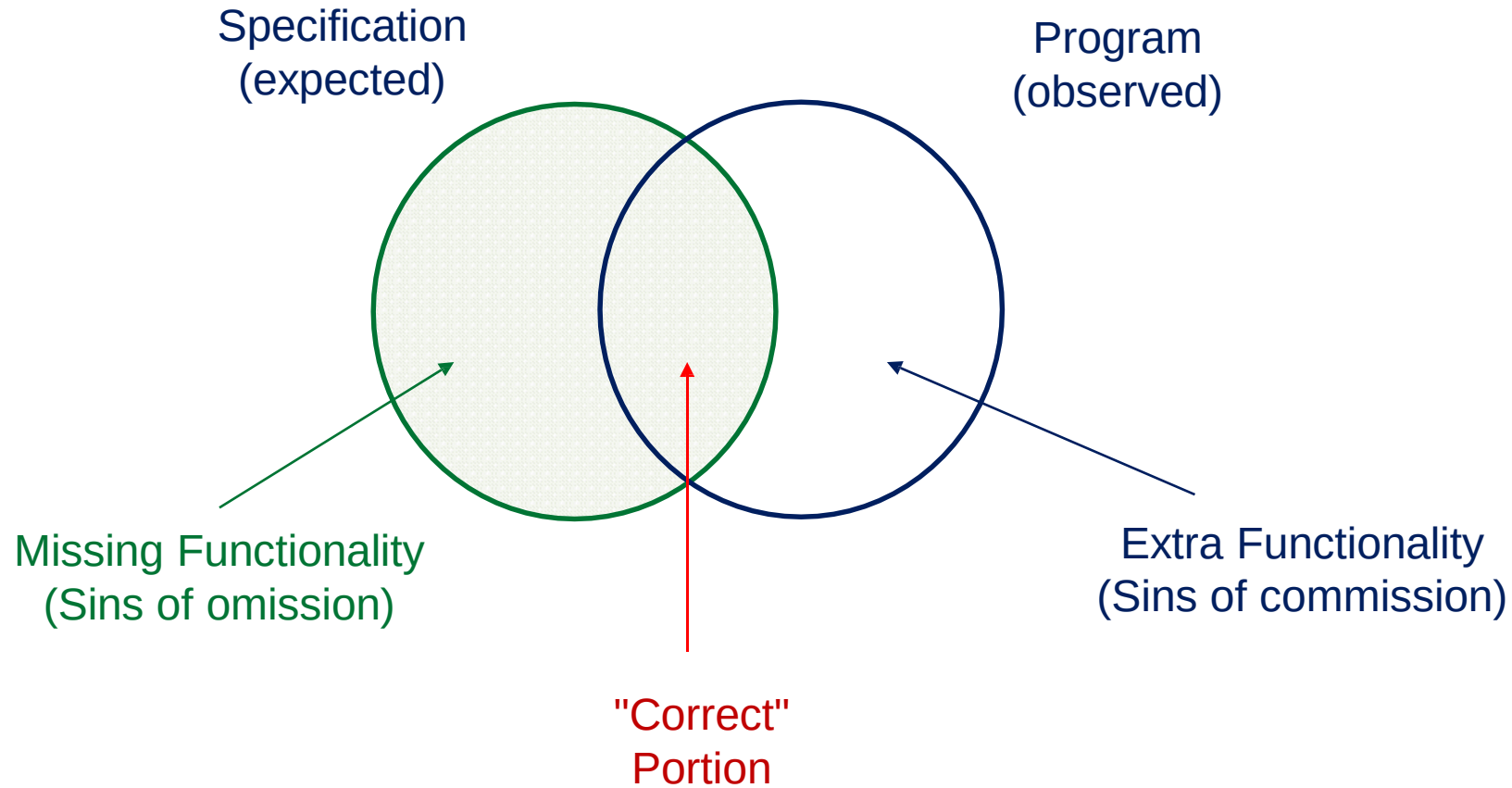
Test Process Analysis

- On the reports
- To improve the application's performance by implementing new technology and additional features

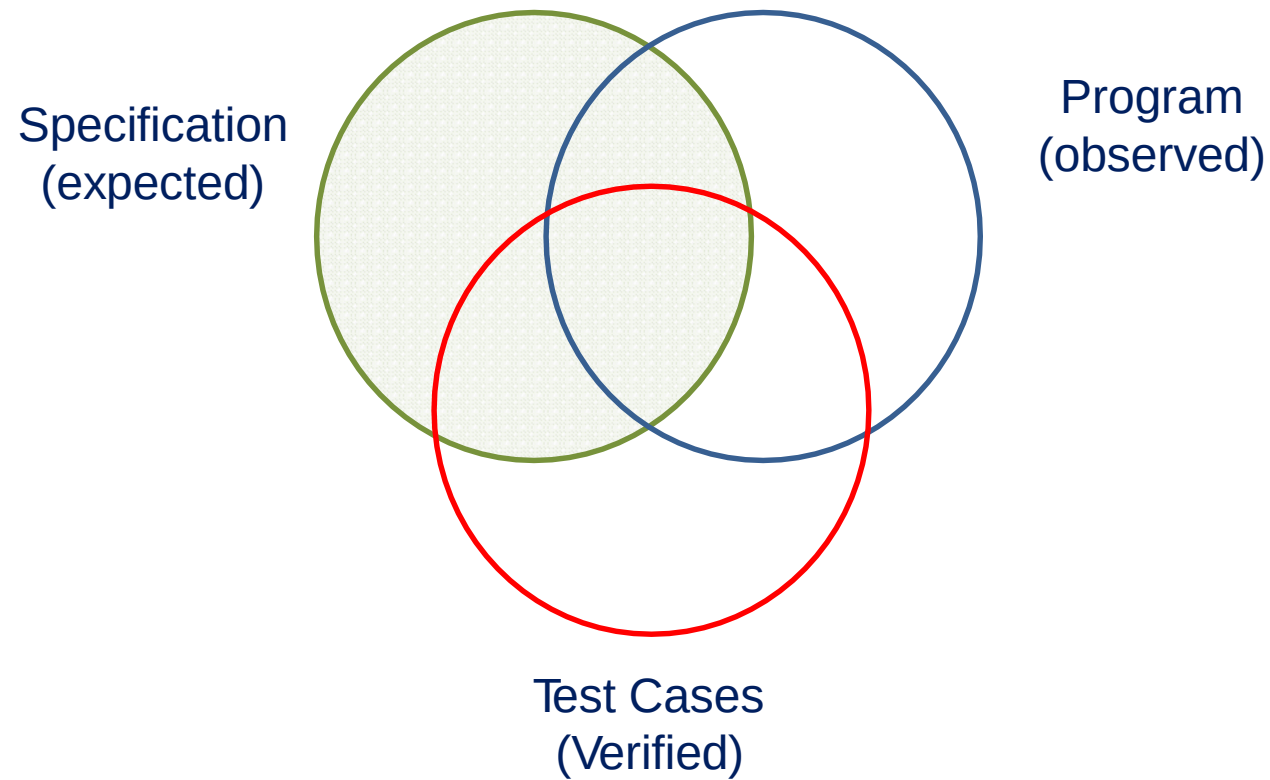
A Strategy for Testing Conventional Software



Program Behaviors



Program Behavior and Test Cases



LET'S GET TESTING

Testing a Ballpoint Pen

- Does the pen **write** in the **right color**, with the **right line thickness**?
- Is the **logo** on the pen according to **company standards**?
- Is it **safe** to **chew** on the pen?
- Does the **click-mechanism** still work **after 100 000 clicks**?
- Does it **still write** after a **car** has **run over it**?



Testing a Ballpoint Pen

- Does the pen write in the right color, with the right line thickness?
- Is the look of the pen according to company standards?
- Is it safe to chew on the pen?
- Does the click-mechanism survive 100 000 clicks?
- Does it still write after a car has run over it?

What is expected from this pen?



Intended use!!

Testing a Software

- Software testing is not that different from testing in other disciplines
 - **Bridges:** verify requirements, design, construction
 - **Cars:** verify requirements, design, construction
 - **Software:** verify requirements, design, implementation



But software is often much harder!



Testing a Software



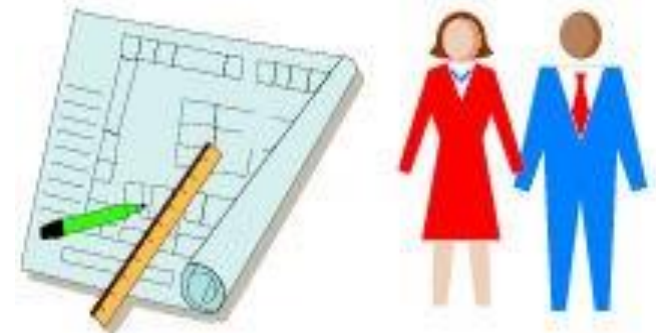
Requirements Definition
Requirements Specification



Functional Requirements
Nonfunctional Requirements



38 Code = System



Design Specification

TESTING APPROACHES

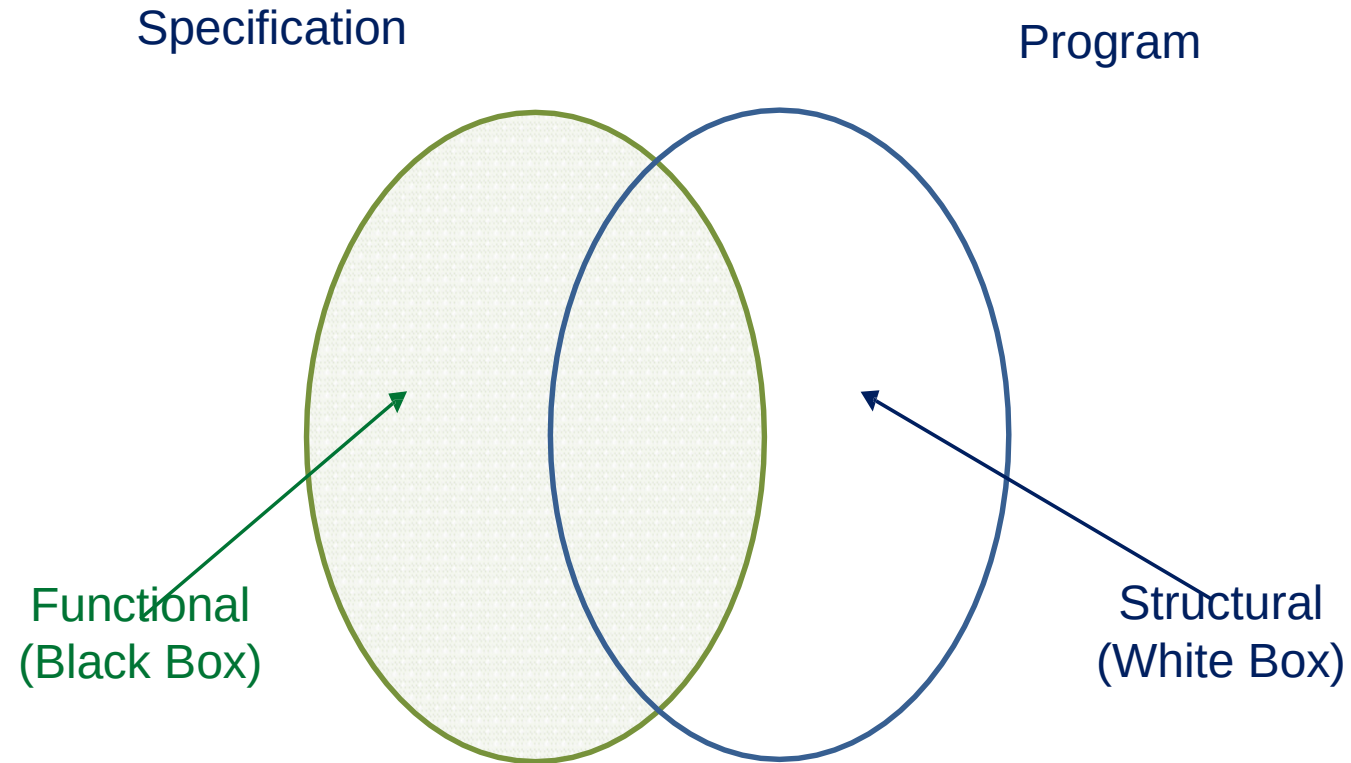
Testing Levels

- Unit testing
 - Testing individual functions, classes and modules
 - Does not catch faults related to interactions
- Integration testing
 - Test interactions between units
 - Based on specification of how units are expected to communicate
 - Does not catch faults related to entire system

Testing Levels

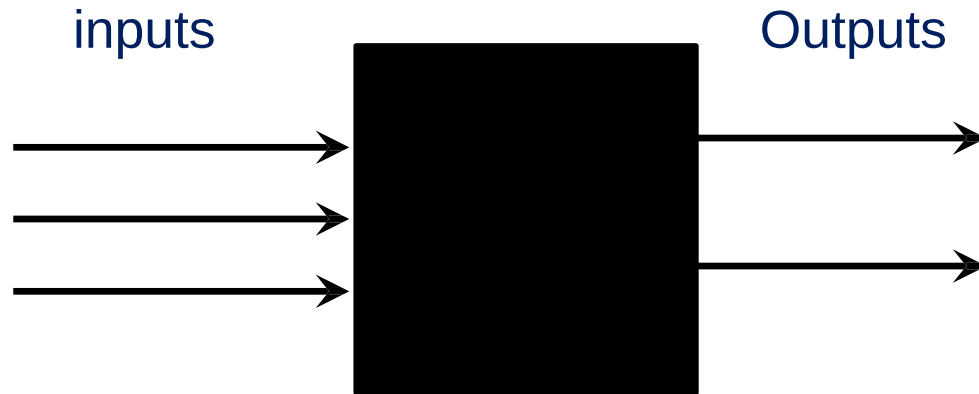
- System testing
 - Tests **entire system as a whole** with focus on **requirements**
 - Can be hard to test exceptional conditions at this level
- Acceptance testing
 - Test that system meets **customer requirements**, often **performed by customer**

Basic Testing Approaches



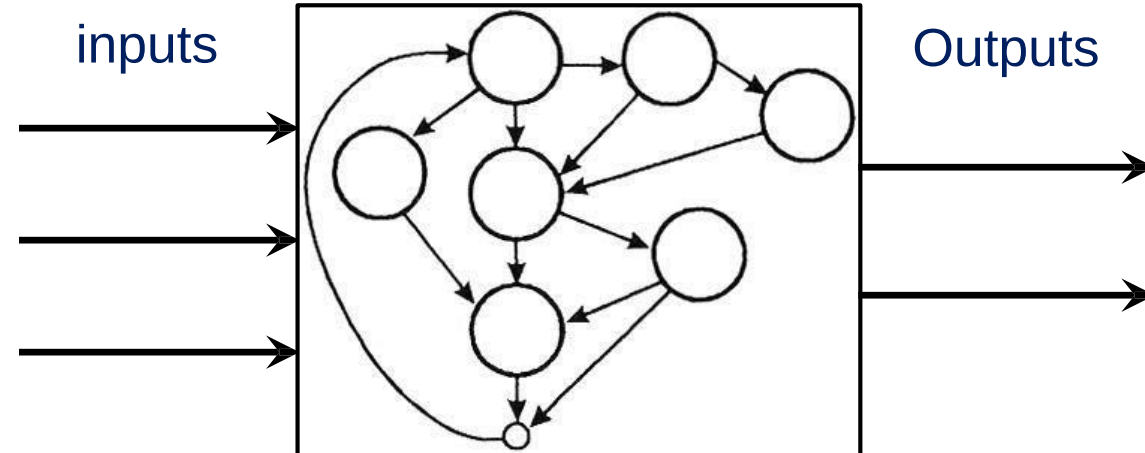
Black Box

- Function is understood only in terms of **it's inputs** and **outputs**, with **no knowledge** of its **implementation**



White Box

- Function is understood only in terms of **its implementation**



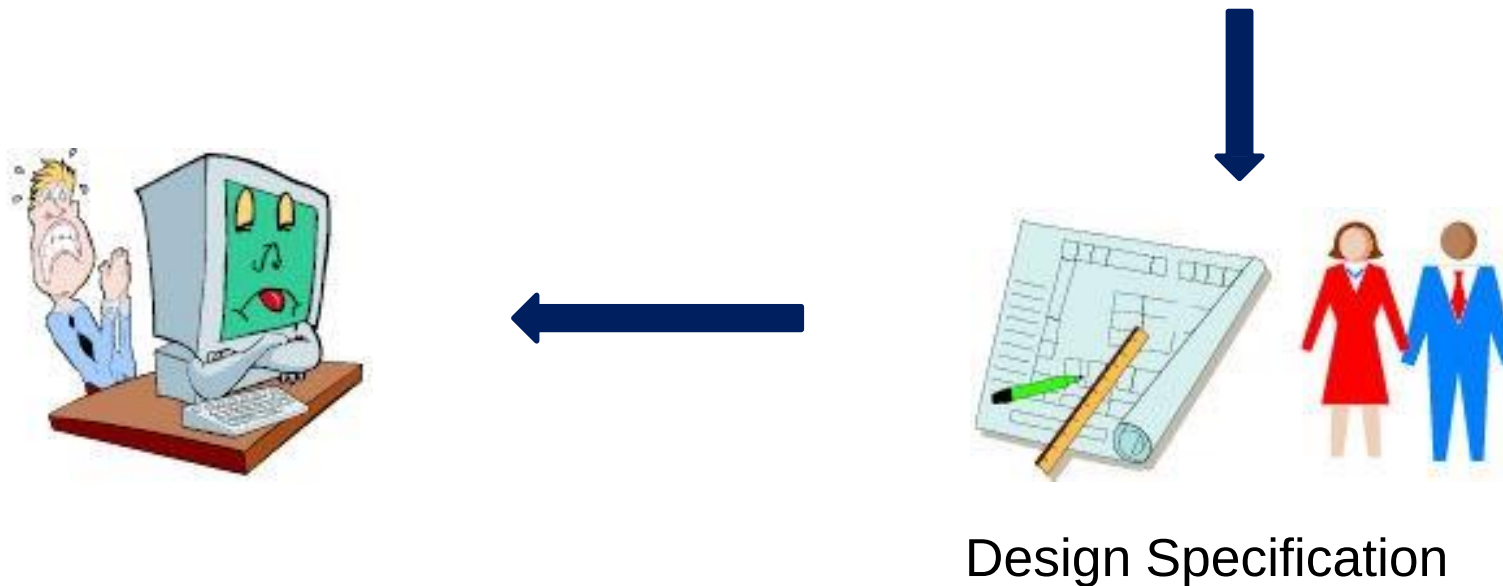
Types (Strategy) of testing

- **Black-box:** A strategy in which testing is based on **requirements** and **specifications**
- **White-box:** A strategy in which testing is also based on **internal paths**, **structure**, and **implementation**
- **Gray-box: ?**

UNIT TESTING

Unit & Integration Testing

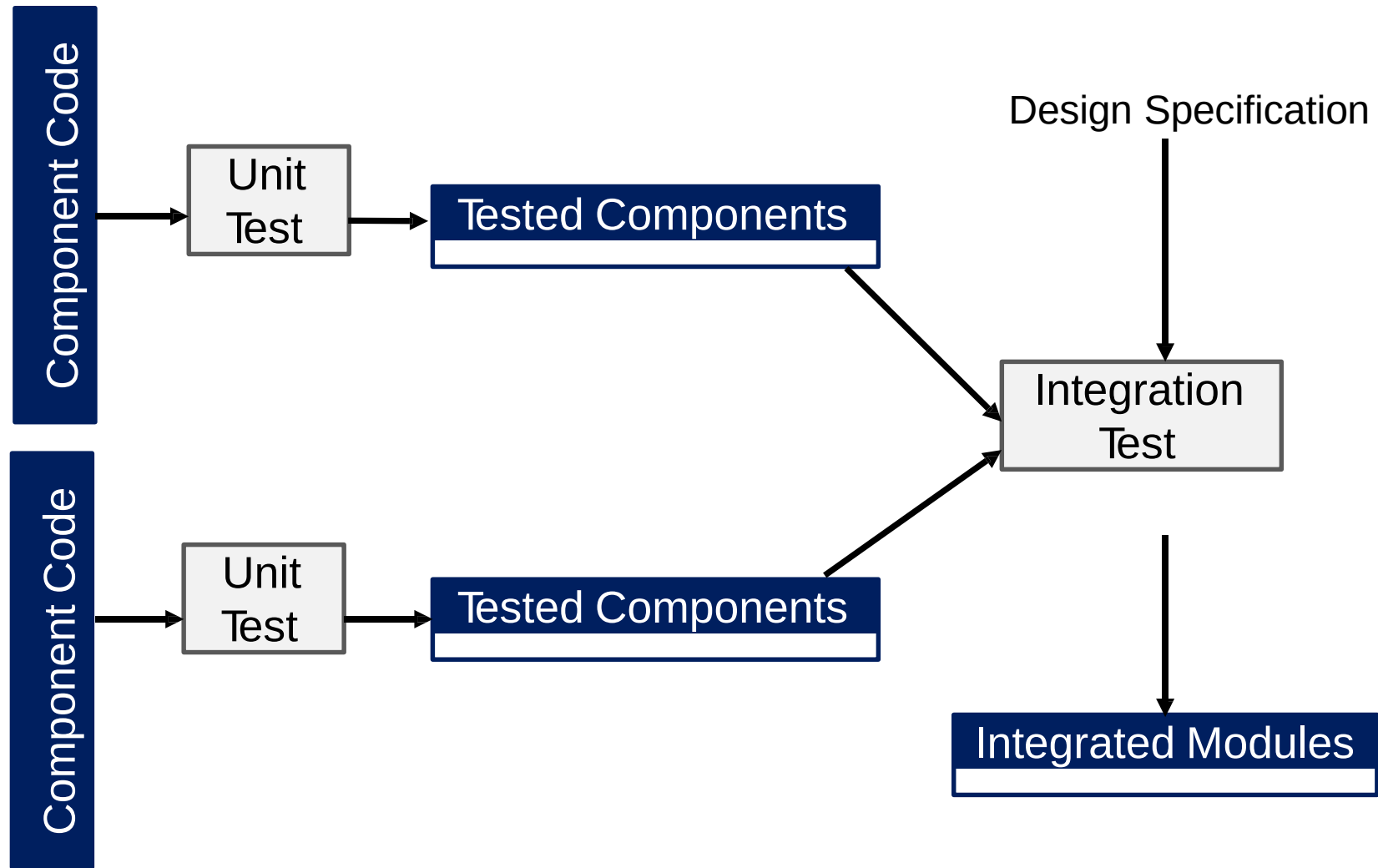
- **Objective:** To ensure that **code** implemented the **design** properly



Unit Testing

- Code Inspections
- Code Walkthroughs
- Black-box Testing
- White-box Testing

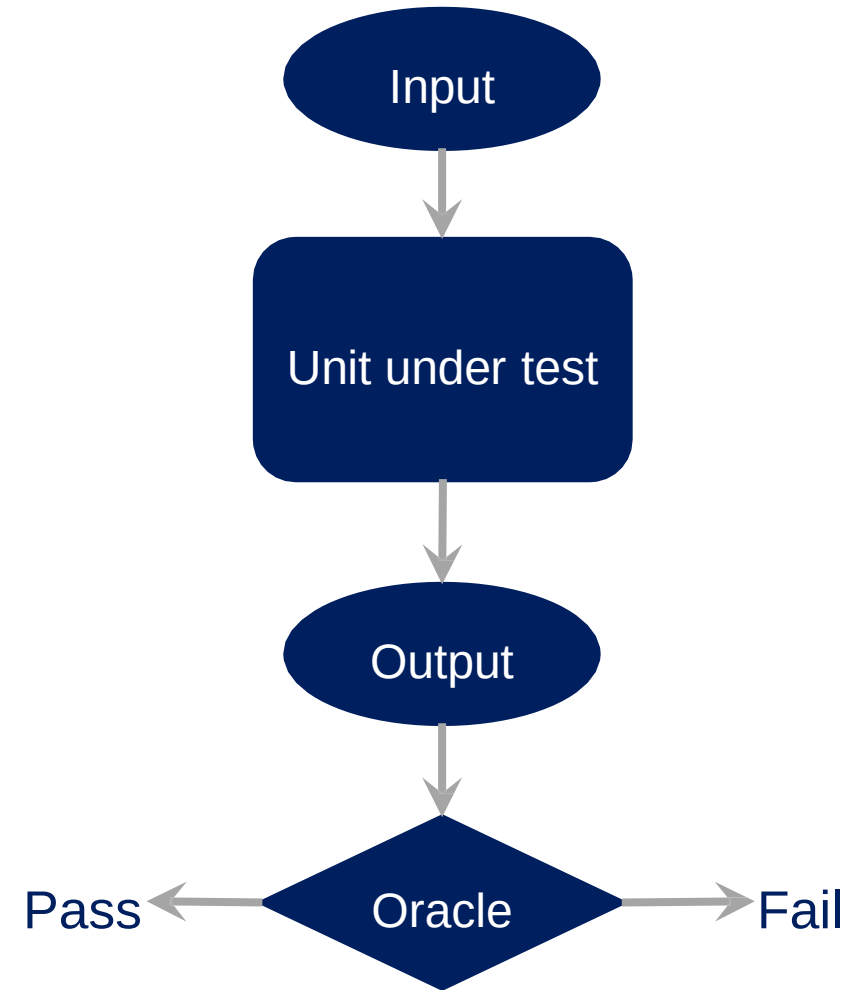
Unit Testing



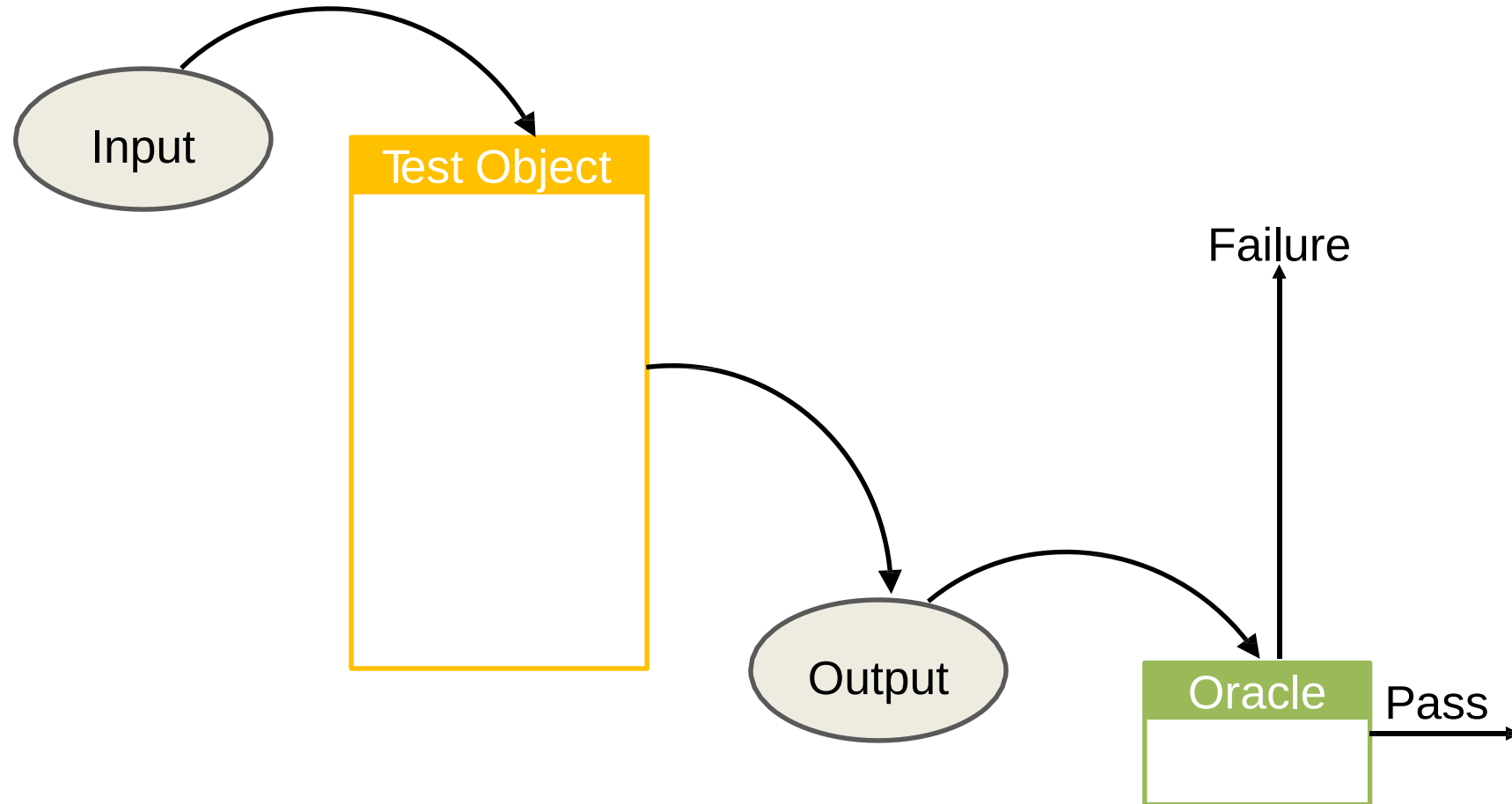
Unit Testing

Different kinds of unit testing

- Code **inspections**
- Code **walkthroughs**
- **Black-box** testing
 - Equivalence class testing
 - Boundary value testing
 - Fuzzy testing
- **White-box** testing
 - Coverage-based testing

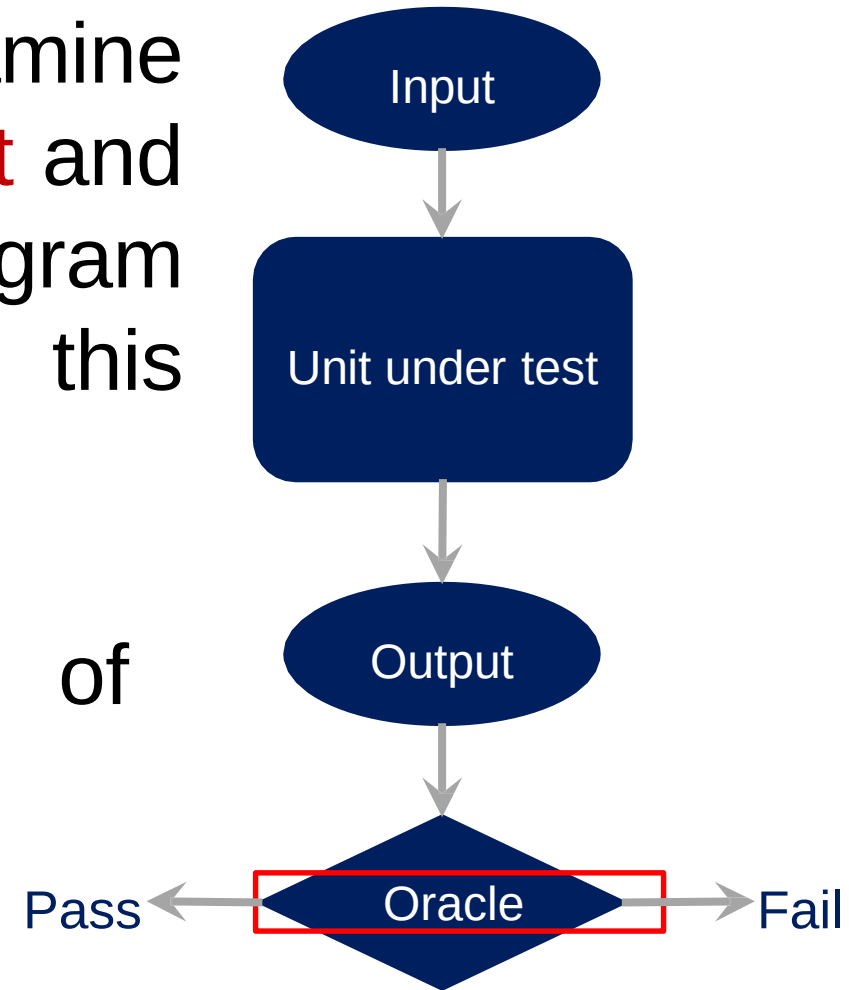


Oracle



Oracles

- **Human:** An **expert** that can examine an **input** and its associated **output** and **determine** whether the program delivered the **correct output** for this **particular input**
- **Automated:** A **system** capable of performing the above task



Recap

- Introduction to software testing
- Definitions and process
- Testing approaches
- Unit testing